

The Harlow Hotel Reservation System

Deliverable 2: Requirements Structuring and System Design

Prepared by: Luis Alfonso, Hannah Baez, Emily Cuberos,
Liennys Fuentes Perez, Emmanuel Prieto

Florida International University

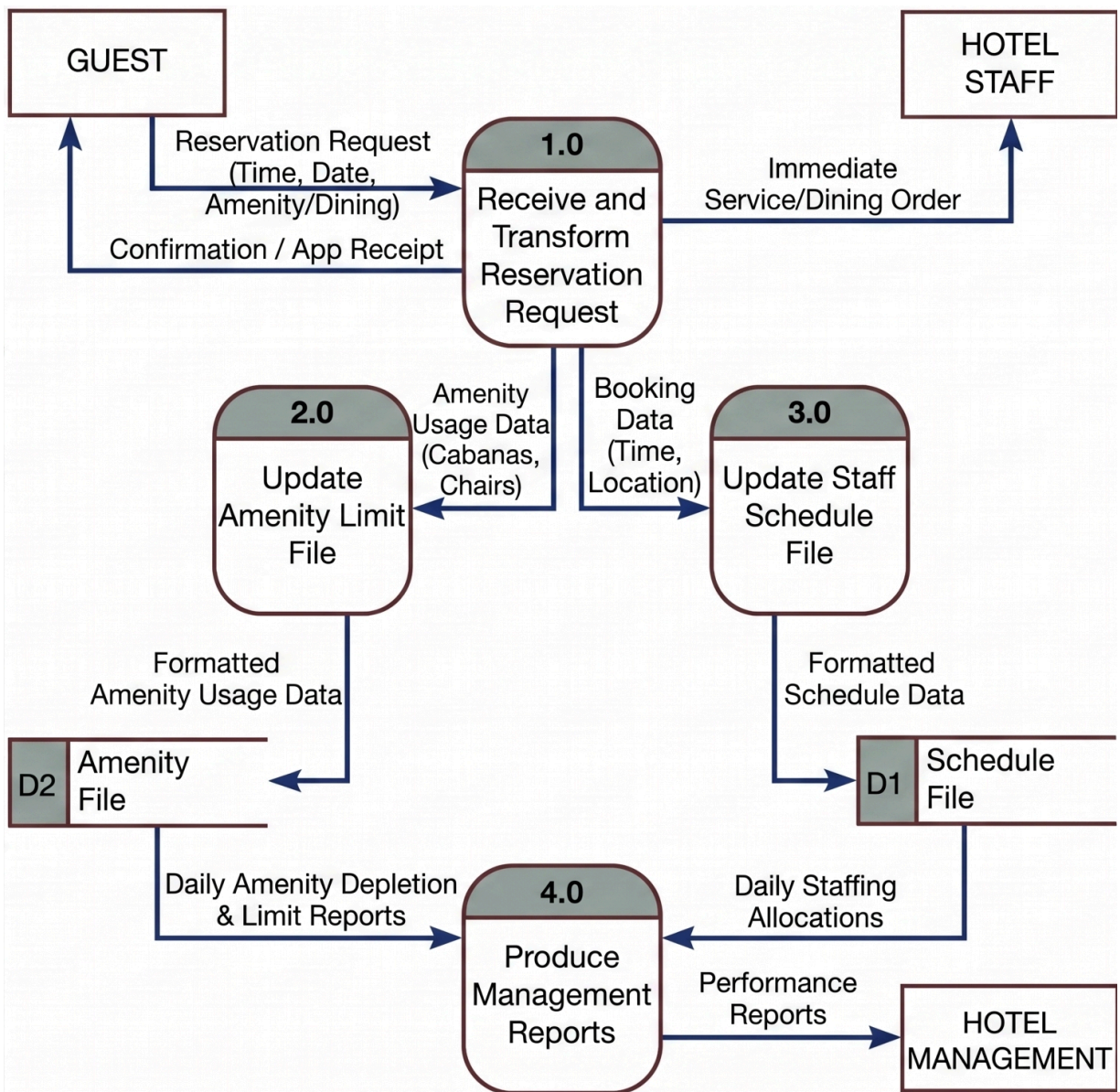
ISM4113: System Analysis and Design

Prepared for: Professor Ricky Sant

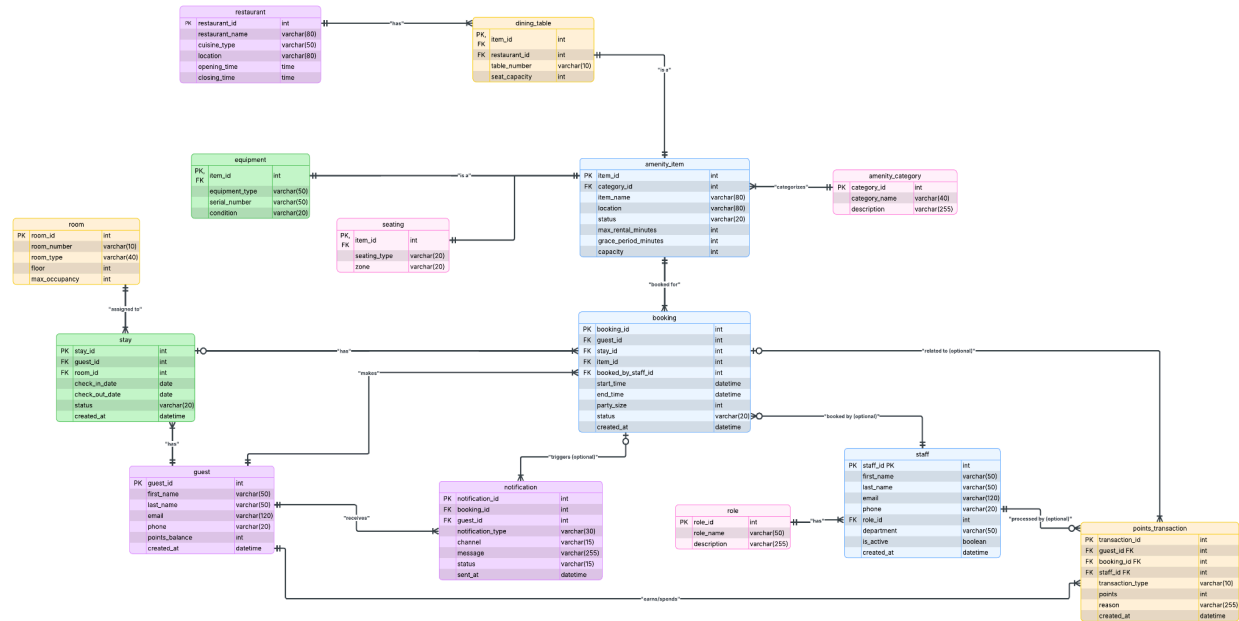
Date: June 14, 2026

The Harlow Hotel Reservation System.....	1
Level-0 DFD Model (Prepared by Luis).....	3
Data Modeling (Prepared by Liennys).....	4
Interface Design (Prepared by Emily).....	5
Data Design (Prepared by Emmanuel).....	5
Database Overview.....	5
Relational Schema.....	6
Data Dictionary.....	7
Normalization (3NF).....	15
Design Decisions.....	15
Security Considerations (Prepared by Hannah).....	16
Account Security and User Authentication.....	16
Reservation Integrity and Double-Booking Prevention.....	17
Protection of Guest and Loyalty Program Data.....	17
Role-Based Access Control.....	17
Data Encryption and Secure Communications.....	18
System Monitoring and Audit Logs.....	18
Physical Security and Staff Device Management.....	18
Backup and Disaster Recovery.....	18
Session Security and Mobile Device Protection.....	19

Level-0 DFD Model *(Prepared by Luis)*



Data Modeling *(Prepared by Liennys)*



Link: https://lucid.app/lucidchart/9893d6d7-22c9-44be-830c-f16b7fe6e2a9/edit?viewport_loc=3385%2C-4000%2C6740%2C3188%2C0_0&invitationId=inv_b0f27d5c-3bb0-4851-acb0-9ac8bb39f71e

This report was formatted by Liennys Fuentes Perez and Hannah Baez based on the collaborative research and data analysis provided by the project group.

Interface Design *(Prepared by Emily)*

App Recording Link:

<https://na01.safelinks.protection.outlook.com/ap/od-59584e83/?url=https%3A%2F%2F1drv.ms%2Fv%2Fc%2Fa95b927067c0df27%2F1QcITCg5PrFySo-VGSRleertAdPSXN8EqWLE0dYXTdQyfxw%3Fe%3DM2oT0I&data=05%7C02%7C%7C40512102b1384757029108dec59b7017%7C84df9e7fe9f640afb435aaaaaaaaaaaa%7C1%7C0%7C639165468554571347%7CUnknown%7CTWfpbGZsb3d8eyJFbXB0eU1hcGkiOnRydWUsIIYiOiIwLjAuMDAwMCIsIIAiOiJXaW4zMlIsIkFOIjoiTWfPbCIIsIldUIjoyfQ%3D%3D%7C0%7C%7C%7C&sdata=WvTLZG4dhje3stGvbbNKdc0zl9UodbwAn696kP4mCBE%3D&reserved=0>

Data Design *(Prepared by Emmanuel)*

This section contains the translation of the system requirements to the database that the booking application will use. It lays out the relational schema (tables, columns and keys that relate these tables to one another) and the initial design for the database. This creates a single centralized database that provides guests and staff with up-to-date availability at the same time and makes it structurally impossible to have double- bookings. This is normalized to Third Normal Form (3NF). This works hand in hand with the Entity Relationship Diagram from the data modeling section.

Database Overview

The database contains 14 tables organized into four groups:

- **People and access:** ROLE, STAFF, and GUEST define who uses the system and what each staff role can see (role-based access control).
- **Stay context:** ROOM and STAY tie each guest to a room and a date range, so a guest can only book amenities between check-in and check-out.

This report was formatted by Liennys Fuentes Perez and Hannah Baez based on the collaborative research and data analysis provided by the project group.

- **Bookable inventory:** AMENITY_CATEGORY and AMENITY_ITEM, the three subtype tables DINING_TABLE, EQUIPMENT, and SEATING, plus RESTAURANT for dining.
- **Transactions:** BOOKING (the core table), NOTIFICATION, and POINTS_TRANSACTION.

Each bookable item (be it a dining table, kayak, cabana) is represented by a single row in AMENITY_ITEM, which contains common fields. It then "branches" into exactly one of the subtype tables which contain the fields unique to that subtype of item. This allows BOOKING to make reservations on any amenity using a single foreign key, while still ensuring the real-time availability/double booking prevention logic is only coded once. The cost is an extra join to read out the full details of an item, which we don't mind paying at this scale.

Relational Schema

Primary keys are marked PK; foreign keys are marked FK.

ROLE (PK role_id, role_name, description)

STAFF (PK staff_id, first_name, last_name, email, phone, FK role_id, department, is_active, created_at)

GUEST (PK guest_id, first_name, last_name, email, phone, points_balance, created_at)

ROOM (PK room_id, room_number, room_type, floor, max_occupancy)

STAY (PK stay_id, FK guest_id, FK room_id, check_in_date, check_out_date, status, created_at)

AMENITY_CATEGORY (PK category_id, category_name, description)

AMENITY_ITEM (PK item_id, FK category_id, item_name, location, status, max_rental_minutes, grace_period_minutes, capacity)

RESTAURANT (PK restaurant_id, restaurant_name, cuisine_type, location, opening_time, closing_time)

DINING_TABLE (PK/FK item_id, FK restaurant_id, table_number, seat_capacity)

EQUIPMENT (PK/FK item_id, equipment_type, serial_number, condition)

SEATING (PK/FK item_id, seating_type, zone)

BOOKING (PK booking_id, FK guest_id, FK stay_id, FK item_id, FK booked_by_staff_id, start_time, end_time, party_size, status, created_at)

NOTIFICATION (PK notification_id, FK booking_id, FK guest_id, notification_type, channel, message, status, sent_at)

POINTS_TRANSACTION (PK transaction_id, FK guest_id, FK booking_id, FK staff_id, transaction_type, points, reason, created_at)

This report was formatted by Liennys Fuentes Perez and Hannah Baez based on the collaborative research and data analysis provided by the project group.

Data Dictionary

Each table lists its columns, data types, key roles, whether the field can be empty (Null), and the constraint or meaning. Primary-key rows are shaded orange; foreign-key rows are shaded green.

ROLE

Defines staff access levels for role-based access control (RBAC).

Column	Data Type	Key	Null	Constraint / Description
role_id	INT	PK	NO	Unique identifier. Auto-increment.
role_name	VARCHAR(50)		NO	Role title. UNIQUE. e.g. Front Desk, Restaurant Host, Manager, IT Admin.
description	VARCHAR(255)		YES	What the role can access on the staff dashboard.

STAFF

Hotel employees who use the staff override the dashboard.

Column	Data Type	Key	Null	Constraint / Description
staff_id	INT	PK	NO	Unique identifier. Auto-increment.
first_name	VARCHAR(50)		NO	Staff first name.
last_name	VARCHAR(50)		NO	Staff last name.
email	VARCHAR(120)		NO	Login email. UNIQUE.
phone	VARCHAR(20)		YES	Contact number.
role_id	INT	FK	NO	References ROLE(role_id).
department	VARCHAR(50)		YES	e.g. Pool & Beach, Restaurant, Front Desk.
is_active	BOOLEAN		NO	TRUE if employed. Default TRUE.

This report was formatted by Liennys Fuentes Perez and Hannah Baez based on the collaborative research and data analysis provided by the project group.

created_at	DATETIME		NO	Record creation timestamp. Default now.
-------------------	----------	--	----	---

GUEST

Hotel guests who book amenities through the app.

Column	Data Type	Key	Null	Constraint / Description
guest_id	INT	PK	NO	Unique identifier. Auto-increment.
first_name	VARCHAR(50)		NO	Guest first name.
last_name	VARCHAR(50)		NO	Guest last name.
email	VARCHAR(120)		NO	Account email. UNIQUE.
phone	VARCHAR(20)		YES	Contact number for notifications.
points_balance	INT		NO	Current compensation points. Default 0. CHECK >= 0.
created_at	DATETIME		NO	Account creation timestamp. Default now.

ROOM

Physical guest rooms on property.

Column	Data Type	Key	Null	Constraint / Description
room_id	INT	PK	NO	Unique identifier. Auto-increment.
room_number	VARCHAR(10)		NO	Displayed room number. UNIQUE.
room_type	VARCHAR(40)		YES	e.g. Standard, Suite, Ocean View.
floor	INT		YES	Floor number.
max_occupancy	INT		YES	Max guests allowed. CHECK > 0.

This report was formatted by Liennys Fuentes Perez and Hannah Baez based on the collaborative research and data analysis provided by the project group.

STAY

A guest's reservation of a room for a date range. Bounds when a guest may book amenities.

Column	Data Type	Key	Null	Constraint / Description
stay_id	INT	PK	NO	Unique identifier. Auto-increment.
guest_id	INT	FK	NO	References GUEST(guest_id).
room_id	INT	FK	NO	References ROOM(room_id).
check_in_date	DATE		NO	First night of stay.
check_out_date	DATE		NO	Departure date. CHECK > check_in_date.
status	VARCHAR(20)		NO	CHECK IN ('Booked','CheckedIn','CheckedOut','Cancelled').
created_at	DATETIME		NO	Record creation timestamp. Default now.

AMENITY_CATEGORY

Lookup table for the three bookable categories.

Column	Data Type	Key	Null	Constraint / Description
category_id	INT	PK	NO	Unique identifier. Auto-increment.
category_name	VARCHAR(40)		NO	UNIQUE. 'Dining','Sports Equipment','Seating'.
description	VARCHAR(255)		YES	Notes on the category.

AMENITY_ITEM

Parent (supertype) table for every bookable item: one specific table, piece of gear, or seat/cabana.

This report was formatted by Liennys Fuentes Perez and Hannah Baez based on the collaborative research and data analysis provided by the project group.

Column	Data Type	Key	Null	Constraint / Description
item_id	INT	PK	NO	Unique identifier. Auto-increment.
category_id	INT	FK	NO	References AMENITY_CATEGORY(category_id).
item_name	VARCHAR(80)		NO	Display name. e.g. 'Cabana 4','Kayak 2','Table 12'.
location	VARCHAR(80)		YES	Physical location on property.
status	VARCHAR(20)		NO	CHECK IN ('Available','Booked','Maintenance','Out OfService'). Default 'Available'.
max_rental_minutes	INT		YES	Max booking length for high-demand items. NULL = no limit.
grace_period_minutes	INT		YES	Minutes before a no-show auto-cancels. NULL = none.
capacity	INT		YES	People the item holds.

RESTAURANT

The three on-property restaurants.

Column	Data Type	Key	Null	Constraint / Description
restaurant_id	INT	PK	NO	Unique identifier. Auto-increment.
restaurant_name	VARCHAR(80)		NO	Restaurant name. UNIQUE.
cuisine_type	VARCHAR(50)		YES	e.g. Italian, Seafood.
location	VARCHAR(80)		YES	Where on property.

This report was formatted by Liennys Fuentes Perez and Hannah Baez based on the collaborative research and data analysis provided by the project group.

opening_time	TIME		YES	Daily open time.
closing_time	TIME		YES	Daily close time.

DINING_TABLE

Subtype of AMENITY_ITEM. Fields for dining tables only.

Column	Data Type	Key	Null	Constraint / Description
item_id	INT	PK, FK	NO	References AMENITY_ITEM(item_id). Also the PK (1:1).
restaurant_id	INT	FK	NO	References RESTAURANT(restaurant_id).
table_number	VARCHAR(10)		NO	Table label within the restaurant.
seat_capacity	INT		NO	Seats at the table. CHECK > 0.

EQUIPMENT

Subtype of AMENITY_ITEM. Fields for sports equipment only.

Column	Data Type	Key	Null	Constraint / Description
item_id	INT	PK, FK	NO	References AMENITY_ITEM(item_id). Also the PK (1:1).
equipment_type	VARCHAR(50)		NO	e.g. Kayak, Paddleboard, Beach Bike.
serial_number	VARCHAR(50)		YES	Asset tag. UNIQUE.
condition	VARCHAR(20)		YES	CHECK IN ('New','Good','Fair','Retired').

SEATING

Subtype of AMENITY_ITEM. Fields for pool/beach seating only.

Column	Data Type	Key	Null	Constraint / Description
item_id	INT	PK, FK	NO	References AMENITY_ITEM(item_id). Also the PK (1:1).
seating_type	VARCHAR(20)		NO	CHECK IN ('Chair','Cabana','Lounge').
zone	VARCHAR(20)		NO	CHECK IN ('Pool','Beach').

BOOKING

Core table. One row per reservation of an amenity item by a guest.

Column	Data Type	Key	Null	Constraint / Description
booking_id	INT	PK	NO	Unique identifier. Auto-increment.
guest_id	INT	FK	NO	References GUEST(guest_id).
stay_id	INT	FK	NO	References STAY(stay_id). Ties booking to an active stay.
item_id	INT	FK	NO	References AMENITY_ITEM(item_id). What is booked.
booked_by_staff_id	INT	FK	YES	References STAFF(staff_id). Set for staff walk-in bookings; NULL if guest self-booked.
start_time	DATETIME		NO	Reservation start.

This report was formatted by Liennys Fuentes Perez and Hannah Baez based on the collaborative research and data analysis provided by the project group.

end_time	DATETIME		NO	Reservation end. CHECK > start_time.
party_size	INT		YES	Number of people (dining). CHECK > 0.
status	VARCHAR(20)		NO	CHECK IN ('Confirmed', 'Check edIn', 'Completed', 'C ancelled', 'NoShow', ' Overdue'). Default 'Confirmed'.
created_at	DATETIME		NO	Booking creation timestamp. Default now.

NOTIFICATION

Confirmations, cancellations, and staff alerts tied to bookings.

Column	Data Type	Key	Null	Constraint / Description
notification_id	INT	PK	NO	Unique identifier. Auto-increment.
booking_id	INT	FK	YES	References BOOKING(booking _id). NULL for non-booking alerts.
guest_id	INT	FK	YES	References GUEST(guest_id). Recipient if guest-facing.
notification_type	VARCHAR(30)		NO	CHECK IN ('Confirmation', 'Can cellation', 'Reminder' , 'OverdueAlert').
channel	VARCHAR(15)		NO	CHECK IN ('Push', 'Email').
message	VARCHAR(255)		YES	Notification body.
status	VARCHAR(15)		NO	CHECK IN ('Pending', 'Sent', 'Fail

This report was formatted by Liennys Fuentes Perez and Hannah Baez based on the collaborative research and data analysis provided by the project group.

				ed'). Default 'Pending'.
sent_at	DATETIME		YES	When delivered. NULL until sent.

POINTS_TRANSACTION

Ledger of compensation points credited or spent by guests.

Column	Data Type	Key	Null	Constraint / Description
transaction_id	INT	PK	NO	Unique identifier. Auto-increment.
guest_id	INT	FK	NO	References GUEST(guest_id).
booking_id	INT	FK	YES	References BOOKING(booking_id). The booking that triggered it, if any.
staff_id	INT	FK	YES	References STAFF(staff_id). Who issued a manual credit. NULL if automatic.
transaction_type	VARCHAR(10)		NO	CHECK IN ('Credit','Debit').
points	INT		NO	Points moved. CHECK > 0.
reason	VARCHAR(255)		YES	Why points were issued or spent.
created_at	DATETIME		NO	Transaction timestamp. Default now.

Normalization (3NF)

The schema is normalized to Third Normal Form: data is stored once, in the right place, with no repeating groups and no fields that depend on anything other than the table's key.

- First Normal Form: every column holds a single value. A party of four is one **BOOKING** row with `party_size = 4`, not four values in one cell.
- Second Normal Form: every non-key column depends on the whole primary key. Single-column keys mean no partial dependencies.
- Third Normal Form: no column depends on another non-key column. A restaurant's name lives only in **RESTAURANT**, not copied into every dining table or booking.

Lookup tables (**ROLE**, **AMENITY_CATEGORY**) and the supertype/subtype split keep category-specific and role-specific data out of the core transaction tables, which prevents update anomalies.

Design Decisions

- **Single **BOOKING** table for all three categories**

Why: Real-time availability and the double-booking freeze run through one place, matching the requirement for a single source of truth.

Tradeoff: Full item detail needs a join to the subtype table; negligible at this scale.

- **Supertype **AMENITY_ITEM** with subtypes**

Why: Shared fields sit once in the parent; unique fields sit in each child, so adding a new category later is low-risk.

Tradeoff: Slightly more tables to maintain than one flat table.

- ****STAY** table between **GUEST** and **BOOKING****

Why: Enforces that a guest can only book between check-in and check-out, a stated requirement.

Tradeoff: One more table, but it is what makes the date-window rule enforceable.

- ****ROLE** table for access control**

This report was formatted by Liennys Fuentes Perez and Hannah Baez based on the collaborative research and data analysis provided by the project group.

Why: Supports role-based access: a restaurant host sees only dining; an IT admin sees everything. Roles change without touching staff records.

Tradeoff: An extra join to resolve permissions.

- **POINTS_TRANSACTION as a ledger**

Why: Every credit and debit is auditable, giving the launch-glitch compensation feature a paper trail. The running total is cached in GUEST.points_balance for fast reads.

Tradeoff: The cached balance must be kept in sync with the ledger by the application.

- **Status fields with CHECK constraints**

Why: Restricting status to a fixed list blocks bad data and powers the overdue-gear and grace-period auto-cancel rules.

Tradeoff: Adding a new status later means updating the constraint.

Security Considerations *(Prepared by Hannah)*

Security is a **critical** aspect of The Harlow Hotel reservation system because the application will manage guest accounts, dining reservations, sports equipment rentals, beach and pool seating reservations, loyalty point balances, and hotel operational data. Since the system replaces several manual processes and third-party reservation services, it must provide a secure and reliable environment for both guests and hotel employees.

Account Security and User Authentication

All guests will be required to create an account before making reservations through the application. User accounts will contain personal information such as names, email addresses, reservation history, and loyalty point balances. To protect this information, passwords should be stored using secure hashing and encryption methods rather than plain text. The system should also require strong password standards and provide secure password recovery procedures.

Hotel employees, managers, and administrators should have separate login credentials with access privileges based on their job responsibilities. Multi-factor authentication should be

This report was formatted by Liennys Fuentes Perez and Hannah Baez based on the collaborative research and data analysis provided by the project group.

considered for management and administrative accounts because these users have access to operational reports, staffing information, and reservation management functions.

Reservation Integrity and Double-Booking Prevention

One of the primary objectives of the project is to eliminate reservation conflicts and double bookings. Therefore, the system must validate availability in real time before confirming any reservation. Once a guest reserves a cabana, beach chair, sports equipment item, or restaurant table, that inventory must immediately update throughout the system.

Database locking and transaction controls should be implemented to ensure that two guests cannot reserve the same resource simultaneously. These controls reduce the possibility of reservation conflicts and help maintain guest trust in the system.

Protection of Guest and Loyalty Program Data

The application will maintain loyalty point balances that may be awarded as compensation when reservation issues occur. Because points have value and can be redeemed throughout the property, the system must protect against unauthorized point additions, deletions, or transfers. All point transactions should be recorded within an audit log showing the user, transaction amount, date, and reason for the adjustment. This creates accountability and helps prevent fraud by guests or employees.

Role-Based Access Control

Different system users require different levels of access. Guests should only be able to view and manage their own reservations. Restaurant employees should only have access to dining reservations. Pool and beach staff should only access amenity reservations relevant to their departments. Managers should have access to operational reports and performance metrics, while administrators maintain system-level controls.

Restricting access according to job responsibilities helps reduce the risk of unauthorized changes and protects sensitive guest information.

Data Encryption and Secure Communications

All communication between the mobile application, hotel tablets, and database servers should be encrypted using HTTPS and SSL/TLS protocols. This prevents attackers from intercepting reservation information while data is being transmitted across public networks. Sensitive information stored in the database, including guest contact information and loyalty account details, should also be encrypted to reduce the impact of a potential data breach.

System Monitoring and Audit Logs

The system should maintain detailed logs of important activities, including account creation, reservation confirmations, cancellations, loyalty point transactions, employee actions, and administrative changes. These logs help hotel management investigate disputes, identify misuse, and monitor overall system performance.

Automated alerts should notify administrators when unusual activity occurs, such as repeated failed login attempts, large point adjustments, or unexpected reservation cancellations.

Physical Security and Staff Device Management

The reservation system will be accessed through hotel-issued tablets used by restaurant hosts, pool attendants, beach staff, and front desk employees. Because these devices may be used throughout the property, physical security controls are necessary to prevent unauthorized access.

All hotel-issued devices should require employee authentication, automatic screen locking after periods of inactivity, and remote device management capabilities. If a device is lost or stolen, administrators should be able to remotely disable access and remove hotel data. Regular software updates should also be enforced to reduce security vulnerabilities and ensure continued system reliability.

Backup and Disaster Recovery

Because hotel operations depend on reservation availability, system downtime could negatively affect guest experiences and hotel revenue. Automated backups should be performed daily and stored securely in a separate location. A disaster recovery plan should also be

This report was formatted by Liennys Fuentes Perez and Hannah Baez based on the collaborative research and data analysis provided by the project group.

established to restore operations quickly if the system experiences hardware failures, software errors, cyberattacks, or natural disasters.

Session Security and Mobile Device Protection

Guests and employees may access the system using smartphones, tablets, and hotel-owned devices. To reduce the risk of unauthorized access, user sessions should automatically expire after a period of inactivity. Hotel tablets should require employee authentication before use, and lost or stolen devices should be capable of remote access removal.

By implementing these security controls, The Harlow Hotel can protect guest information, maintain reservation accuracy, reduce operational risk, and provide a secure and reliable booking experience that supports the overall goals of the project.